Titanic 二分类模型学习笔记

1.总体说明

1.1 文件说明

```
titanic_submission/
├── src/
│   ├── main.py             #主程序
│   ├── data.py             #数据处理
│   ├── model.py            #模型类
│   ├── train.py            #具体训练
│   ├── plot.py             #画图
│   ├── predict.py          #预测函数
│   ├── predict_new.py      #具体预测某个新数据
│   └── requirements.txt    #依赖库
│
├── docs/
│   ├── 学习笔记.md
│   ├── 学习笔记.pdf
│   ├── final/              #最终版运行结果
│   │   ├── model.pth
│   │   ├──loss_curve.png
│   │   └──accuracy_curve.png
│   └── runs/               #调试结果都放在了这里，要改一下if false，否则覆盖final
│
├── draft/                  #最初版，一大段代码
│   └── titanic_draft.py
│
└── train.csv
```

1.2 任务目标

根据乘客的特征信息，最终输出遇难或幸存两个结果：

- 0:遇难
- 1:幸存

1.3 大致流程

```
数据读取
→ 数据划分
→ 数据清洗
→ 特征工程
→ 类别特征数值化
→ 标准化
→ Tensor 转换
→ Dataset / DataLoader
```

→ 构建模型
→ 训练
→ 测试
→ 保存模型
→ 新乘客预测

1.4 设备

本人尝试过把模型搬到gpu上跑（草稿就是在gpu上），但是感觉比在cpu上跑还慢，查询原因说是对于小数据集来说，数据传输可能花费的时间可能比计算花费的时间还要长，于是正式版并未加入to（device）

二、数据读取与划分

2.1 数据读取

原数据文件是表格，所以很适合pandas爬取，使用pandas读取 `train.csv`，结果如下（pd.read_csv）：

共891条数据

共12列，详细字段见题干

其中`Survived`是预测标签

2.2 划分训练集和测试集

将数据按照 8:2 划分为训练集和测试集

训练集：

- 样本数：712

测试集：

- 样本数：179

使用 `train_test_split()` 完成划分

2.3 关于验证集

在一轮中已经详细学习，其实按理说加一个验证集才更合理，但是本题主要考察建模能力，因此只按题干要求划分了测试集和训练集

2.4 stratify

划分数据时使用 `stratify=y`，使训练集和测试集中的0/1类别比例尽可能与原始数据保持一致，相当于分层抽样

如果没有这个，那可能划分训练集测试集的时候会出现这种极端情况：

幸存者全到了训练集，遇难者全到了测试集，训练的时候全输出1，百分之百的准确率，但是一到测试集，全部预测错误

所以需要使用stratify=y

2.5 随机种子

本人在这里吃过很大的亏，正式版一开始就没弄种子，导致运行了几次发现不一样，吓坏了，后面赶紧补救了，总而言之需要固定的地方又一下几个地方：

- `train_test_spilt()`

  需要random_state=seed，sklearn自带的random模块

- `dataloader`

  不同于pytorch全局随机种子`torch.manual_seed()`，dataloader还会在全局种子上在加工重新弄一个随机数，所以需要专门的`generator`来弄

- pytorch参数初始化

  这个就是上面提到的`torch.manual_seed()`，用来初始化模型参数的

将这几个种子固定下来了，整个实验基本就可以复现了，如果还有用到其他模块（`np.random.seed ()`，python自带的`random.seed()`等等），也需要自行固定种子

三、数据清洗-缺失值

缺失值的查看可以用`print(df.isna().sum())`

3.1 Age

缺失177条，这个建议填入中值，因为均值受极端值影响太大了，缺失的太多也不好用众数填，所以一律用训练集的中位数填充

3.2 Embarked

缺失2个，很少很少，填进去众数就行了

3.3 Cabin

缺失687条，占比77%，很多了，这往往意味着有没有cabin也是一个比较重要的事情，可以考虑构建新特征has_cabin，至于cabin不为空的，可以考虑提取第一个字母，毕竟那个甲板那个房间太细了，索性直接看甲板，没有的直接填入`Unknown`，剩下的在合并只有几个数字的甲板为rare，就差不多了

四、数据清洗-数值化与特征工程

4.1 Sex

只有两类，可以直接按如下规则处理：

- male→0
- female→1

4.2 Title

文件里的name列很整齐，姓氏，称谓·名字，称谓也反映了某个人的社会地位，或许能作为参照，所以用正则匹配出每个人的称谓（具体如何匹配的在草稿的注释里），然后将同义的合并，再将少见的归为rare，便构造出来了有一个特征

4.3 Ticket

这个票号跟PassengerID一样只是编号，但是这个能反映一个票有多少人在用，或许也能纳入模型考量，于是有了`ticket_group_size`这一特征，具体实现在草稿里

这里面还有一个问题我觉得比较重要，就是同一个票号，假如其名下有4个人，可能3个分到了训练集，1个分到了测试集，如果严格的防止数据泄露，那么这个票号应该算3个人，但是实际不是这样的，如果按实际来，那就会数据泄露，虽然感觉没有泄露标签问题应该不大，但是肯定不希望数据泄露，所以最后还是决定按照题目要求来，严格按照训练集处理

4.4 family_size

这个不同于票号下面的人数，一个家庭可能用好多票，也有可能好多家庭用一个票号，所以关于家人也要单独构造

`family_size=SibSp+Parch+1`(SibSp,两个s大写坑了我好几次)，注意加上本人

4.5 is_alone

根据 family_size 判断乘客是否独自一人：

- family_size = 1 → 1
- family_size > 1 → 0

这个特征用于进一步表达乘客是否有同行家人

五、数据清洗-独热编码

5.1 为什么要独热编码

对于多类别且类别之间没有明显距离关系的数据，不能用简单的1，2，3，4编码，否则某个神经元可能会学到距离关系，像是什么北京大于石家庄，重庆大于成都，都会被当作特征计算进去，但是事实显然不是这样，于是编码成空间向量，他们的距离就相同了

5.2 具体怎么独热编码

在草稿中有具体过程，主要使用`pd.get_dummies()`编码'embarked'，'title'，以及'deck'

5.3 reindex

就拿embarked举例，训练集可能有c，q，s，编码完返回：

- embarked_c
- embarked_d
- embarked_q

但是测试集可能没有q，导致和训练集不一样，没法预测了就，所以要reindex重新对齐并填入值，这一点我理解了好久

六、标准化

6.1 为什么标准化

在一轮中有详细说明，主要是平衡数据尺度，防止某些大尺度数据给整个模型拉偏

七、dataframe→numpy→tensor→dataset→dataloader

7.1 dataframe

前面其实一直都在用pandas的dataframe表格，主要是方便，像是按列操作，删除，命名，编码什么的，pandas里都有相应的函数

7.2 numpy

这一步在这个项目里面不是必须项，但是通常项目里都会这么转换，将numpy里的数组作为常见的中间桥梁，可以进行一些矩阵运算什么的，因为是纯数组所以转换成其他形式也很方便（比如tensor）

7.3 tensor

pytorch主要接收tensor来进行运算处理，所以还要转换成这个

7.4 dataset

dataset主要是解决'怎么拿单个数据'，定义dataset类的时候就给了个`__getitem__`方法，相当于给个索引拿数据和标签

7.5 dataloader

将数据一个个的喂给模型当然是可行的，但是一是慢，二是好像容易过拟合

```
epoch:100 , loss:0.2607 , train_accuracy:0.9031 , test_accuracy:0.7877
```

这是我把batch_size弄成1给出来的结果，不如成批算的快，结果也没有成批算的乐观

（ps：关于成批算怎么算，主要是矩阵乘法，分别计算损失再**平均**得到整体损失，再依据这个梯度来反向更新）

八、模型设计

8.1 模型选择

具体情况具体分析，这个模型最后输入的是26个特征，输出1个标签，linear很适合干这个事情，把多个数据压缩成少量数据，但是也不能光线性变化，于是又每一层引入了relu非线性变化，层层交织就自然的组成了一个简单的mlp（多层感知机），所以我的模型里面是linear+relu

8.2 损失函数

这是一个二分类任务，最后输出一个logit，压缩成sigmoid，预测一个概率，一轮里面我了解到了交叉熵，很适合概率预测的损失计算，引入了log，就很容易放大概率变化导致的结果变化，对应到pytorch里的函数就是

bcewithlogitsloss，把sigmoid和交叉熵弄一块了（防止单算造成的浮点数损失）

8.3 优化器

依据梯度更新参数的东西，我在一轮里面主要了解到了最简单最经典的sgd，参数减去学习率乘梯度得到新参数，但是这样很容易有问题，学习率是统一的，但是梯度不是，可能两个变量的梯度数量级差别很大，使得每次乘相同的学习率，更新的幅度很不一样，于是就有了adam优化，公式如下：

$$g_t = \nabla_\theta L(\theta_t)$$

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon}$$

- θ，模型参数
- g_t，梯度
- m_t，一阶矩，梯度方向的平均趋势，毕竟把之前的梯度也算了进去
- β，‘记住’的权重
- v_t，二阶矩，记录梯度的大小
- $\hat{m}_t, \hat{v}_t$，偏差修正，一开始的m_0, v_0都是零，会吃掉太多权重，所以补回来，利用等比数列求和
- α，即是lr，学习率
- ϵ，一个很小的数，防止分母变为零（如果$\hat{v}_t$为零，那么上面的$\hat{m}_t$一般也会为零，所以很少出现突然变得很大的时候）
- $\frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon}$，上面控制方向，下面控制幅度，$\hat{m}_t$一样的话，$\hat{v}_t$大的，代表近期梯度很陡峭，那就应该慢一点，放在分母里就很容易表达这个意思

九、模型训练

9.1 超参数设置

- 学习率和轮次：

  初始是0.001，能够正常训练，最后一轮（100）结果是`epoch:100 , loss:0.3498 , train_accuracy:0.8876 , test_accuracy:0.8101`

  后面改成0.0001，模型就卡住了（依旧100轮）`epoch:100 , loss:0.5132 , train_accuracy:0.6166 , test_accuracy:0.6145`

  (从数字上看应该全预测的0)

  但是把轮次调到1000就好点了`epoch:1000 , loss:0.3429 , train_accuracy:0.8834 , test_accuracy:0.8045`

最后还是选定了lr=0.001，epoch=100

中间还有一个现象，在运行日志（runs）里面有几个图是我在实验lr和epoch（10月9号11点左右，具体已标记），里面有反转的现象，尤其是我把lr设置为0.0002，更加印证了我的猜想，步子太小，就得靠轮次堆，要不然就会弄到一个平台被卡住

9.2 隐藏层

这个是在草稿里面实验的，最后选定的是16到8到4到2，实验的是16到8，准确率不如这个，所以没有改动，**这里面还有问题，具体在'新乘客预测'里**，最后改成了（16，8）

十、实际问题

10.1 准确率卡在61%

最一开始没有固定种子，以为是随机性，后面多尝试了几轮，发现还是这样，最后排查模型输出`print(history[probability][:10])`发现由nan，又返回去：

```
print(X_train.isna().sum())
print(X_test.isna().sum())
```

发现是is_alone的问题，最后看代码是自动补全给我的：

```
X_test['is_alone']=(X_test['family_size']==1).astype(int)
```

后面的test弄成了train，索引对不上，全填了nan，后面改回来了，发现还在报错

10.2 类型错误

报错说keyerror，给我说了是哪一行，看了看，发现这一句：

```
X_test['ticket_group_size']=X_test['ticket_group_size'].fillna(1)
```

自动补全给我弄成了：

```
X_test['ticket_group_size']=X_test['Ticket'].fillna(1)
```

导致后面标准化的时候弄成了字符串运算，难受

10.3 自动补全

好多报错说是找不到键或是变量，顺着给的行数去查，都是自动补全把我的变量名改了或者没填对

10.4 其他

前面还涉及到一些问题，像是ticket_group_size的取舍，还有超参数的设置等等，在前面有具体回答

十一、保存模型

这一点的草稿我写在了正式版里，因为草稿里并没有涉及到这个保存，还是属于碰见问题解决问题式的学习

11.1 路径

学习了path模块，使之能够迁移

11.2 final与run

写了个if语句，切换保存路径，true为运行，保存在runs里（用时间命名的），false为保存终版，存在final里面

11.3 模型的保存

官方推荐保存参数，这样也确实灵活，所以我弄在了字典里，加载的时候还得在重新写一下模型

（另：我保存的是测试集准确率最高的那一版）

十二、新乘客预测

这块的代码注释也在正式版里，也是加载模型与参数然后数据处理（由代码注释也能看出来是完完全全按照训练集处理的）最后预测

12.1 构造新乘客

```
new_passenger=pd.DataFrame(
[{
    "PassengerId": 1001,
    "Pclass": 3,
    "Name": "Smith,Mr.Mary",
    "Sex": "male",
    "Age": 29,
    "SibSp": 4,
    "Parch": 2,
    "Ticket": "NEW 001",
    "Fare": 80.0,
    "Cabin": "B85",
    "Embarked": "C"
}]
)
```

12.2 预测

第一次跑这个模型预测，很正常，输出一个或1，还有具体的概率（0.5581081509590149，我不会忘记这个数字的），但是当我尝试修改乘客，比如换个性别什么的，发现还是这个数字，换了修改pclass，变成0.4857759177684784了，说明模型能响应一些变量，但是后来我又尝试更改特征，又回到了0.5581081509590149，既然特征确确实实改变了，那么就只能看看是不是计算中有点问题，具体草稿留在了正式版里，发现如果层数太深，那么relu就会把一些特征直接弄成0，导致后面的计算也没了意义，所以最后我改了模型层数，变成了（16，8）

十三、总结

其实也没什么好作结的，因为具体的学到的代码都在草稿里，剩下的一些宏观思路和具体问题的分析都在上面了，总而言之，这是我第一次做这种项目，问题还有不少，相信会在日后的学习里逐渐改进弥补